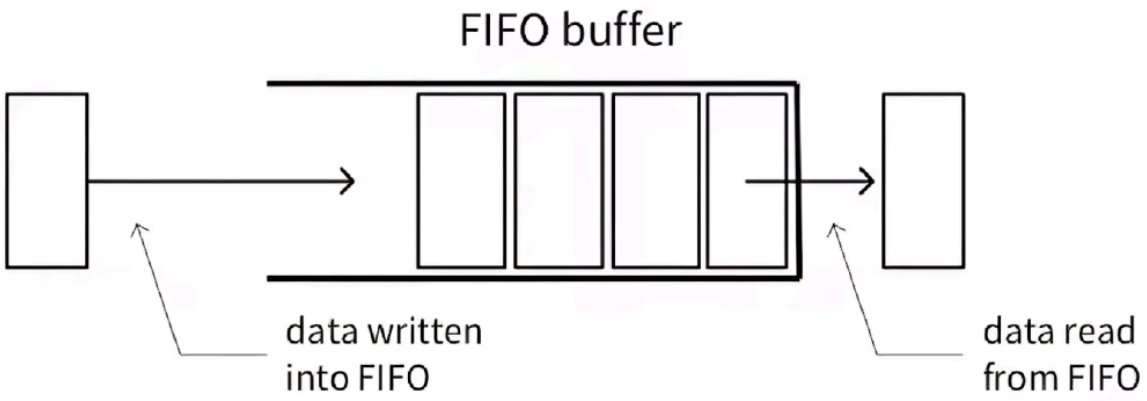
Synchronous FIFO Design & Verification Project

Submitted by : Rawan Mansour Abdelrahman

Project Description



This project aims to design and verify a **FIFO (First In First Out)** memory using **SystemVerilog**.
The FIFO stores data in order — the first written data is the first to be read — and includes control signals for reading, writing, and status flags such as `full`, `empty`, `almostfull`, and `almostempty`.

Verification is done using **SystemVerilog Assertions (SVA)** placed inside the design to check that the FIFO works correctly in all situations.
Assertions automatically monitor the internal signals and ensure correct pointer updates, counter behavior, and flag generation during operations like read, write, overflow, and underflow.

In addition, **functional coverage** is used to confirm that all features and possible states of the FIFO have been tested, by covering different combinations of control inputs and status flags.

FIFO Design – Bug Report

Bug Description	Expected Behavior	Fix Summary
1. `wr_ack` and `overflow` signals are not properly reset or controlled. These signals do not get cleared on reset, and `wr_ack` may be asserted even when both read and write occur simultaneously.	On reset, both signals should be deasserted. `wr_ack` should only be asserted when a valid write occurs (FIFO not full), and `overflow` should assert only when a write is attempted while the FIFO is full.	Add reset conditions for `wr_ack` and `overflow`. Modify logic so that `overflow` and `wr_ack` only change based on correct FIFO states.
2. `underflow` is declared as a combinational signal instead of sequential.	The `underflow` signal must be updated on the rising edge of the clock and cleared during reset.	Move the `underflow` logic inside a clocked always block with reset control.
3. Incorrect conditions for `full` and `almostfull`. Current logic uses `(count == FIFO_DEPTH)` for `full` and `(count == FIFO_DEPTH - 2)` for `almostfull`.	`full` should assert when the number of stored elements equals the FIFO depth, and `almostfull` should assert when only one write can occur before the FIFO becomes full (`count == FIFO_DEPTH - 1`).	Fix the threshold logic for `full` and `almostfull`.
4. Missing handling for simultaneous read and write (`wr_en` & `rd_en` both high). The design ignores the `2'b11` case in the counter logic.	When both read and write are high: if FIFO empty → write only; if FIFO full → read only; else → perform both (count remains unchanged).	Add a case in the counter logic for `{wr_en, rd_en} == 2'b11` with the proper behavior.

Bug Description	Expected Behavior	Fix Summary
5. Reset is incomplete. Not all internal signals (like `data_out`, `rd_ptr`, `count`, `overflow`, `wr_ack`) are reset when `rst_n` is low.	During reset, all sequential signals and pointers should be initialized to zero.	Add full reset conditions for all sequential variables inside each always block.
6. Missing pointer wrap-around. The write and read pointers keep incrementing beyond the FIFO depth.	After the pointer reaches the last entry, it should wrap back to zero.	Add conditional wrap-around logic: `ptr <= (ptr == FIFO_DEPTH-1) ? 0 : ptr + 1;`

Verification Plan

Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	Check that all internal signals and flags reset correctly. FIFO should start empty with all outputs cleared.	Directed at the start of the simulation	–	Immediate assertion to verify async reset
FIFO_2	Verify write operation increases count and advances write pointer properly when FIFO not full.	Randomized during the simulation	–	Concurrent assertion to check `count` and `wr_ptr` update
FIFO_3	Verify read operation decreases count and moves read pointer to next element when FIFO not empty.	Randomized during the simulation	–	Concurrent assertion to check `count` and `rd_ptr` update
FIFO_4	When both read and write are active, FIFO performs both correctly without affecting count.	Randomized during the simulation	–	Concurrent assertion to verify stable count and pointer movement
FIFO_5	When invalid operations happen (write on full / read on empty), count and pointers must remain constant.	Randomized during the simulation	–	Concurrent assertion to ensure no updates on invalid operations
FIFO_6	The full flag should assert exactly when FIFO reaches its maximum capacity.	Randomized and boundary-based stimulus	Cover bins for count equal to FIFO_DEPTH	Assertion to verify correct full flag activation
FIFO_7	The empty flag should assert when FIFO is completely empty.	Randomized and directed stimulus	Cover bins for count = 0	Assertion to verify correct empty flag activation
FIFO_8	The almostfull flag should assert when FIFO is one location away from full.	Randomized boundary stimulus	Cover bin for count = FIFO_DEPTH-1	Assertion to verify correct almostfull flag
FIFO_9	The almostempty flag should assert when there is only one element left.	Randomized boundary stimulus	Cover bin for count = 1	Assertion to verify correct almostempty flag
FIFO_10	Overflow flag asserts for one clock when write occurs while full.	Randomized overflow scenario	Cover overflow events	Sequential assertion to verify overflow behavior
FIFO_11	Underflow flag asserts for one clock when read occurs while empty.	Randomized underflow scenario	Cover underflow events	Sequential assertion to verify underflow behavior
FIFO_12	When pointers reach the last address, they must wrap to zero correctly.	Directed wraparound test	Cover pointer wrap events	Assertion to verify wraparound behavior
FIFO_13	Counter must never exceed FIFO depth or go below zero.	Randomized full-range test	Cover bins at counter limits	Assertion to verify valid counter range

RTL Code

DO file

```
vlib work
vlog *.sv  +cover -covercells +define+SIM
vsim -voptargs=+acc work.top -cover
coverage save FIFO.ucdb -onexit -du FIFO
do wave.do
run -all

vcover report FIFO.ucdb -details -annotate -all -output Report.txt
```

1. FIFO Design

```
module FIFO(IF.DUT Design);

parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;

localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

// Write Logic: handles wr_ack and overflow
always @(posedge Design.clk or negedge Design.rst_n) begin
   if (!Design.rst_n) begin
       wr_ptr <= 0;
       Design.wr_ack <= 0;
       Design.overflow <= 0;
   end
   else if (Design.wr_en && !Design.full) begin
       mem[wr_ptr] <= Design.data_in;
       Design.wr_ack <= 1;
       // wrap around when pointer reaches end
       wr_ptr <= (wr_ptr == FIFO_DEPTH-1) ? 0 : wr_ptr + 1;
   end
   else begin
       Design.wr_ack <= 0;
       // overflow asserted if write attempted while FIFO full
       if (Design.wr_en && Design.full)
           Design.overflow <= 1;
       else
           Design.overflow <= 0;
   end
end

// Read Logic: handles data_out and underflow
always @(posedge Design.clk or negedge Design.rst_n) begin
   if (!Design.rst_n) begin
       rd_ptr <= 0;
       Design.data_out <= 0;
       Design.underflow <= 0;
   end
   else if (Design.rd_en && !Design.empty) begin
       Design.data_out <= mem[rd_ptr];
       // wrap around when pointer reaches end
       rd_ptr <= (rd_ptr == FIFO_DEPTH-1) ? 0 : rd_ptr + 1;
       Design.underflow <= 0;
   end
   else begin
       // underflow asserted if read attempted while FIFO empty
       if (Design.rd_en && Design.empty)
           Design.underflow <= 1;
       else
           Design.underflow <= 0;
   end
end

// Counter Logic: handles simultaneous read/write correctly
always @(posedge Design.clk or negedge Design.rst_n) begin
   if (!Design.rst_n) begin
       count <= 0;
   end
   else begin
       case ({Design.wr_en, Design.rd_en})
```

```
            2'b10: if (!Design.full)  count <= count + 1;       // write only
            2'b01: if (!Design.empty) count <= count - 1;       // read only
            2'b11: begin                                        // both high
                if (Design.empty)      count <= count + 1;      // empty    write only
                else if (Design.full)  count <= count - 1;      // full     read only
                else                   count <= count;          // normal    no change
            end
            default: count <= count;
        endcase
    end
end

// Flag Assignments
assign Design.full = (count == FIFO_DEPTH)? 1 : 0;
assign Design.empty = (count == 0)? 1 : 0;
assign Design.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;  // BUG: FIFO_DEPTH-1 not FIFO_DEPTH-2
assign Design.almostempty = (count == 1)? 1 : 0;

`ifdef SIM

// Concurrent Assertion
property BothActive_StableCount;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.wr_en && Design.rd_en && !Design.full && !Design.empty)
  |=> $stable(count) &&
      ((wr_ptr == $past(wr_ptr) + 1) || (($past(wr_ptr) == FIFO_DEPTH-1) && wr_ptr == 0)) &&
      ((rd_ptr == $past(rd_ptr) + 1) || (($past(rd_ptr) == FIFO_DEPTH-1) && rd_ptr == 0));
endproperty

property WriteOnly_CountInc;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.wr_en && !Design.rd_en && !Design.full)
  |=> (count == $past(count) + 1);
endproperty

property ReadOnly_CountDec;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (!Design.wr_en && Design.rd_en && !Design.empty)
  |=> (count == $past(count) - 1);
endproperty

property WriteFull_NoPtrMove;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.wr_en && Design.full)
  |=> $stable(wr_ptr);
endproperty

property ReadEmpty_NoPtrMove;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.rd_en && Design.empty)
  |=> $stable(rd_ptr);
endproperty

BothActiveCheck: assert property(BothActive_StableCount);
WriteIncCheck:   assert property(WriteOnly_CountInc);
ReadDecCheck:    assert property(ReadOnly_CountDec);
WriteFullCheck:  assert property(WriteFull_NoPtrMove);
ReadEmptyCheck:  assert property(ReadEmpty_NoPtrMove);

// Reset Assertions
always_comb begin
    if (!Design.rst_n) begin
        assert final(count == 0);
        assert final(wr_ptr == 0);
        assert final(rd_ptr == 0);
        assert final(Design.full == 0);
```

```
        assert final(Design.empty == 1);
        assert final(Design.almostfull == 0);
        assert final(Design.almostempty == 0);
        assert final(Design.overflow == 0);
        assert final(Design.underflow == 0);
    end
end

// Flag Assertions
always_comb if (count == FIFO_DEPTH)     assert final(Design.full);
always_comb if (count == 0)              assert final(Design.empty);
always_comb if (count == FIFO_DEPTH - 1) assert final(Design.almostfull);
always_comb if (count == 1)              assert final(Design.almostempty);

// Sequential Flag Behavior
property WriteAck_OK;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.wr_en && !Design.full) |=> Design.wr_ack;
endproperty

property Overflow_OK;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.wr_en && Design.full) |=> Design.overflow;
endproperty

property Underflow_OK;
  @(posedge Design.clk) disable iff(!Design.rst_n)
  (Design.rd_en && Design.empty) |=> Design.underflow;
endproperty

property WrPtr_Wrap;
  @(posedge Design.clk)
  (wr_ptr == FIFO_DEPTH-1 && Design.wr_en && !Design.full) |=> (wr_ptr == 0);
endproperty

property RdPtr_Wrap;
  @(posedge Design.clk)
  (rd_ptr == FIFO_DEPTH-1 && Design.rd_en && !Design.empty) |=> (rd_ptr == 0);
endproperty

property Pointers_InRange;
  @(posedge Design.clk)
  (count <= FIFO_DEPTH) && (wr_ptr < FIFO_DEPTH) && (rd_ptr < FIFO_DEPTH);
endproperty

AckCheck:      assert property(WriteAck_OK);
OverflowCheck: assert property(Overflow_OK);
UnderflowCheck:assert property(Underflow_OK);
WrPtrWrap:     assert property(WrPtr_Wrap);
RdPtrWrap:     assert property(RdPtr_Wrap);
PtrRange:      assert property(Pointers_InRange);

`endif

endmodule
```

2. Interface

```
interface IF(input bit clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;

    logic [FIFO_WIDTH-1:0] data_in;
    logic [FIFO_WIDTH-1:0] data_out;
    logic rst_n;
    logic wr_en;
    logic rd_en;
    logic wr_ack;
    logic overflow;
    logic underflow;
    logic full;
    logic empty;
    logic almostfull;
    logic almostempty;

    // Design modport
    modport DUT(
        input clk, rst_n, wr_en, rd_en, data_in,
        output data_out, wr_ack, overflow, underflow, full, empty, almostfull, almostempty
    );

    // Testbench modport
    modport TB(
        output clk, rst_n, wr_en, rd_en, data_in,
        input data_out, wr_ack, overflow, underflow, full, empty, almostfull, almostempty
    );

endinterface
```

3. Transaction

```
package FIFO_transaction_pkg;

    class FIFO_transaction;
        parameter FIFO_WIDTH = 16;

        // Distribution weights
        int RD_EN_ON_DIST = 30;
        int WR_EN_ON_DIST = 70;

        // Random inputs
        rand logic [FIFO_WIDTH-1:0] data_in;
        rand logic rst_n;
        rand logic wr_en;
        rand logic rd_en;

        // Observed outputs
        logic [FIFO_WIDTH-1:0] data_out;
        logic wr_ack;
        logic overflow;
        logic full;
        logic empty;
        logic almostfull;
        logic almostempty;
        logic underflow;

        // Constructor with default weights
        function new(int wr_weight = 70, int rd_weight = 30);
            WR_EN_ON_DIST = wr_weight;
            RD_EN_ON_DIST = rd_weight;
        endfunction

        // Constraints for signal distribution
        constraint control_signals {
            rst_n dist {0 := 1, 1 := 99};
            wr_en dist {1 := WR_EN_ON_DIST, 0 := (100 - WR_EN_ON_DIST)};
            rd_en dist {1 := RD_EN_ON_DIST, 0 := (100 - RD_EN_ON_DIST)};
        }

    endclass

endpackage
```

4. Coverage

```
package FIFO_coverage_pkg;
    import FIFO_transaction_pkg::*;

    class FIFO_coverage;
        FIFO_transaction cov_txn;

        // Covergroup for functional coverage
        covergroup fifo_cg;
            // Input coverpoints
            wr_en_cp: coverpoint cov_txn.wr_en;
            rd_en_cp: coverpoint cov_txn.rd_en;

            // Output coverpoints
            wr_ack_cp: coverpoint cov_txn.wr_ack;
            overflow_cp: coverpoint cov_txn.overflow;
            full_cp: coverpoint cov_txn.full;
            empty_cp: coverpoint cov_txn.empty;
            almostfull_cp: coverpoint cov_txn.almostfull;
            almostempty_cp: coverpoint cov_txn.almostempty;
            underflow_cp: coverpoint cov_txn.underflow;

            // Cross coverage between control signals and status flags
            cross_wr_ack: cross wr_en_cp, rd_en_cp, wr_ack_cp;
            cross_overflow: cross wr_en_cp, rd_en_cp, overflow_cp;
            cross_full: cross wr_en_cp, rd_en_cp, full_cp;
            cross_empty: cross wr_en_cp, rd_en_cp, empty_cp;
            cross_almostfull: cross wr_en_cp, rd_en_cp, almostfull_cp;
            cross_almostempty: cross wr_en_cp, rd_en_cp, almostempty_cp;
            cross_underflow: cross wr_en_cp, rd_en_cp, underflow_cp;

        endgroup

        // Constructor
        function new();
            cov_txn = new();
            fifo_cg = new();
        endfunction

        // Sample transaction data
        function void sample_data(FIFO_transaction txn);
            cov_txn = txn;
            fifo_cg.sample();
        endfunction

    endclass

endpackage
```

5. Scoreboard

```
package FIFO_scoreboard_pkg;
    import shared_pkg::*;
    import FIFO_transaction_pkg::*;

    class FIFO_scoreboard;
        parameter FIFO_WIDTH = 16;
        parameter FIFO_DEPTH = 8;

        // Reference model queue
        logic [FIFO_WIDTH-1:0] fifo_queue [$];
        logic [FIFO_WIDTH-1:0] data_out_ref;

        // Check data against reference model
        function void check_data(FIFO_transaction txn);
            reference_model(txn);

            if(txn.rst_n && txn.rd_en) begin
                if(data_out_ref === txn.data_out) begin
                    correct_count++;
                end else begin
                    error_count++;
                    $display("Time:%0t ERROR: Expected: %h, Got: %h (wr_en=%0b, rd_en=%0b)",
                             $time, data_out_ref, txn.data_out, txn.wr_en, txn.rd_en);
                end
            end
        endfunction

        // Reference model implementation
        function void reference_model(FIFO_transaction txn);
            if(!txn.rst_n) begin
                fifo_queue.delete();
                data_out_ref = 0;
            end
            else begin
                // Write operation
                if(txn.wr_en && (fifo_queue.size() < FIFO_DEPTH)) begin
                    fifo_queue.push_back(txn.data_in);
                end

                // Read operation
                if(txn.rd_en && (fifo_queue.size() > 0)) begin
                    data_out_ref = fifo_queue.pop_front();
                end
            end
        endfunction

        // Constructor
        function new();
            data_out_ref = 0;
        endfunction

    endclass

endpackage
```

6. Monitor

```
module monitor(IF.TB if_mon);

    import shared_pkg::*;
    import FIFO_transaction_pkg::*;
    import FIFO_coverage_pkg::*;
    import FIFO_scoreboard_pkg::*;

    FIFO_coverage coverage_obj;
    FIFO_transaction trans_obj;
    FIFO_scoreboard scoreboard_obj;

    initial begin
        coverage_obj = new();
        trans_obj = new();
        scoreboard_obj = new();

        forever begin
            // Wait for new inputs
            @(pass_inputs);

            // Sample inputs
            trans_obj.data_in = if_mon.data_in;
            trans_obj.rst_n = if_mon.rst_n;
            trans_obj.wr_en = if_mon.wr_en;
            trans_obj.rd_en = if_mon.rd_en;

            // Wait for outputs to stabilize
            @(negedge if_mon.clk);

            // Sample outputs
            trans_obj.data_out = if_mon.data_out;
            trans_obj.wr_ack = if_mon.wr_ack;
            trans_obj.overflow = if_mon.overflow;
            trans_obj.full = if_mon.full;
            trans_obj.empty = if_mon.empty;
            trans_obj.almostfull = if_mon.almostfull;
            trans_obj.almostempty = if_mon.almostempty;
            trans_obj.underflow = if_mon.underflow;

            // Parallel coverage and checking
            fork
                coverage_obj.sample_data(trans_obj);
                scoreboard_obj.check_data(trans_obj);
            join

            // Check for test completion
            if(test_finished) begin
                $display("Time:%0t - Test Complete: Correct=%0d, Errors=%0d",
                         $time, correct_count, error_count);
                $finish;
            end
        end
    end

endmodule
```

7. Testbench

```
module tb(IF.TB if_tb);
    import FIFO_transaction_pkg::*;
    import shared_pkg::*;

    parameter NUM_TESTS = 1000000;
    FIFO_transaction trans_obj;

    initial begin
        trans_obj = new();

        apply_reset();

        for(int i = 0; i < NUM_TESTS; i++) begin
            @(negedge if_tb.clk);
            void'(trans_obj.randomize());
            drive_signals();
            -> pass_inputs;
        end

        test_finished = 1;
    end

    // Reset task
    task apply_reset();
        if_tb.rst_n = 0;
        @(negedge if_tb.clk);
        if_tb.rst_n = 1;
    endtask

    // Drive signals to interface
    task drive_signals();
        if_tb.rst_n = trans_obj.rst_n;
        if_tb.wr_en = trans_obj.wr_en;
        if_tb.rd_en = trans_obj.rd_en;
        if_tb.data_in = trans_obj.data_in;
    endtask

endmodule
```

8. Top Module

```
module top;
    bit clk;

    // Interface instance
    IF top_if(clk);

    // Module instances
    FIFO dut(top_if.DUT);
    monitor mon(top_if.TB);
    tb testbench(top_if.TB);

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Simulation timeout
    initial begin
        #1000000;
        $display("Simulation timeout");
        $finish;
    end

endmodule
```

9. Shared Package

```
package shared_pkg;
    // Synchronization event
    event pass_inputs;

    // Scorekeeping variables
    int error_count = 0;
    int correct_count = 0;

    // Test control
    bit test_finished = 0;
endpackage
```